Contrato Tecnico End-to-End

XelerGo

Validacion contractual entre frontend y backend,
con foco en autenticacion, ordenes, wallets, XCOP,
tiempo real y funcionalidad cripto por proveedor externo.

FECHA DEL ANALISIS	21 de marzo de 2026
FRONTEND	Vue 3 + TypeScript + Pinia + Vuetify
BACKEND	Laravel + Sanctum + Pusher + servicios de proveedor cripto
OBJETO DEL DOCUMENTO	Verificar si las uniones entre frontend y backend estan realmente cerradas y que brechas siguen abiertas.

Dictamen ejecutivo preliminar

La union observable entre ambas capas es parcial y fragil. Existen contratos funcionales que si responden, pero el ciclo principal de recepcion y envio cripto por proveedor externo no queda correctamente soportado de punta a punta.

Creado por Jose Manuel Guerrero Duarte, Ing en informatica

1. Resumen ejecutivo

Este documento consolida la auditoria previa del frontend disponible en la raiz del proyecto, la auditoria previa del backend localizada en Xelergo-Backend/AUDITORIA_QA_PROYECTO.pdf y la revision directa del codigo necesaria para verificar si el contrato real entre ambas capas esta cerrado o solo aparenta estarlo.

La conclusion operativa es que el acoplamiento entre frontend y backend no esta roto en todos los frentes, pero tampoco constituye una integracion end-to-end madura. Hay alineacion observable en autenticacion inicial, creacion basica de ordenes, provision de ciertas direcciones y transferencia interna de XCOP. Sin embargo, cuando el flujo depende de validacion critica, estados confiables, tiempo real, proveedor externo o seguridad transversal, la union se vuelve parcial, manual o fragil.

Sintesis de decision

* Estado E2E general: cumplimiento parcial y fragil.
* Aprobacion para produccion con foco cripto por proveedor externo: no recomendable.
* Visualmente la plataforma parece mas madura de lo que realmente esta su contrato tecnico.
* El objetivo principal de envio y recepcion de criptomonedas por proveedor externo no queda correctamente soportado end-to-end.

* Lo mejor alineado hoy es la autenticacion inicial, la persistencia de sesion, la creacion estructural de ordenes y la transferencia interna de XCOP dentro del ecosistema.
* Lo que bloquea la aprobacion es la ausencia de un lifecycle confiable de recepcion, confirmacion y desembolso cripto por proveedor externo, sumado a fallas de consumo entre frontend y backend en puntos sensibles.
* Las auditorias previas ya marcaban falta de cierre operativo; la verificacion cruzada actual confirma esa lectura desde la union real entre ambas capas.

2. Alcance y fuentes consideradas

El backend se toma en cuenta solo en lo necesario para validar contratos, payloads, recursos, rutas, guards y eventos que el frontend consume o presupone. Este documento no replica la auditoria backend; la usa como contexto para determinar si la capa de presentacion esta bien unida a la capa de servicio.

Fuentes documentales revisadas: AUDITORIA_QA_FRONTEND_VUE.pdf y Xelergo-Backend/AUDITORIA_QA_PROYECTO.pdf

Fuentes tecnicas contrastadas: Componentes Vue, stores Pinia, helper HTTP, router, guards, composables y vistas criticas del frontend; controladores, requests, resources, rutas, eventos, canales y servicios del backend.

Cobertura contractual priorizada: Autenticacion, confirmaciones sensibles, ordenes compra/venta, wallets, provision de direcciones, XCOP interno, retiro XCOP, tiempo real, flujos administrativos con txid y seguridad transversal.

Limites de validacion

* No se ejecutaron operaciones reales contra BitGo ni contra otros proveedores de cripto.
* No se dispuso de pruebas E2E automaticas que validaran el lifecycle completo desde la UI hasta la liquidacion real.
* Las conclusiones se apoyan en evidencia observable del repositorio, contratos de rutas, recursos de salida y comportamiento deducible del codigo.

3. Metodologia de contraste

* Se trazaron los flujos desde la accion visible del frontend hasta la ruta backend, request validator, controlador y resource que sostienen la respuesta.
* Se contrastaron endpoints consumidos por el frontend con el inventario real de rutas exportado por el backend y con la definicion de routes/api.php.
* Se valido si los datos exigidos por la UI coinciden con lo que el backend persiste, valida y devuelve.
* Se revisaron canales Pusher, autorizacion de broadcasting, guards de navegacion, manejo de tokens, errores, estados manuales y representaciones visuales de lifecycle.
* Se incorporo como contexto la conclusion de la auditoria backend, que ya calificaba el objetivo principal de cripto por proveedor externo como no cumplido.

Como verificacion mecanica adicional se cruzaron 27 endpoints literales multisegmento extraidos del frontend. De ellos, 25 pudieron verificarse directamente contra el inventario de rutas; en esos casos la brecha detectada no fue ausencia de endpoint, sino consumo incorrecto, validacion debil o semantica incompleta.

4. Panorama contractual resumido

Mapa de estado por contrato

* Login inicial y 2FA de acceso: alineado con observaciones.
* Confirmaciones sensibles post-login: parcial, con defecto operativo de consumo.
* Creacion de orden compra/venta: parcial, por validacion insuficiente.
* Wallets y direcciones de deposito: parcial, con abstraccion de proveedor inconsistente.
* Transferencia interna XCOP: alineada solo en alcance interno.
* Retiro XCOP: parcial y asimetrico entre capacidad backend y UI real.
* Estados en tiempo real de ordenes: no alineados de forma confiable.
* Flujo cripto por proveedor externo: no soportado correctamente end-to-end.

5. Contratos observables verificados

5.1 Autenticacion inicial y sesion

Estado: Alineado

* El contrato base de login y 2FA de acceso existe y es consistente entre frontend y backend.

* El frontend hace POST a login y, cuando recibe HTTP 205, abre el modal 2FA y reintenta por login/f2a.
* El backend devuelve 205 cuando el usuario tiene google_authentication activo y entrega plainTextToken cuando la autenticacion se completa.
* El frontend separa el token por el delimitador de Sanctum y almacena solo la parte posterior al caracter |, lo que coincide con el formato devuelto por el backend.
* Este es uno de los pocos contratos criticos que si aparece razonablemente alineado de punta a punta.

Evidencia observable

```
- Xelergo-Backend/app/Http/Controllers/Auth/LoginController.php:61, 77 y 92 devuelve 205 o token plainTextToken segun el flujo.
- src/views/auth/Login.vue:14 define login/f2a; en 32 abre el flujo 2FA cuando llega 205; en 67 separa token por '|'.
- src/stores/TwoFactorAuthStore.ts:85 repite el mismo criterio de almacenamiento del token.
```

5.2 Confirmaciones sensibles post-login y navegacion protegida

Estado: No alineado

* Los contratos de confirmacion por contrasena o 2FA, usados para acciones sensibles, presentan un defecto de consumo y la navegacion protegida del cliente no es deterministica.

* TwoFactorAuthStore usa /users/check/password y /users/verificate/code/2fa con slash inicial, mientras la baseURL del frontend ya contiene /api.
* Con VITE_API_URL configurado como https://v2.xeler.io/api, ese slash inicial puede resolver fuera del prefijo /api y romper confirmaciones sensibles aunque la ruta exista en backend.
* El router principal y middleware personalizados ejecutan next() en ramas que no retornan, por lo que la proteccion visual y la navegacion pueden comportarse de forma inconsistente.
* El backend sigue protegido por sanctum, pero la experiencia del usuario y la capa de autorizacion del cliente quedan fragiles.

Evidencia observable

- .env:2 fija VITE_API_URL=https://v2.xeler.io/api.
- src/helper.ts:51 define baseURL desde la variable de entorno y en 56 agrega Authorization Bearer.
- src/stores/TwoFactorAuthStore.ts:50 y 55 llama /users/check/password y /users/verificate/code/2fa con slash inicial.
- Xelergo-Backend/routes/api.php:149 y 152 registra verificate/code/2fa y check/password bajo el grupo users autenticado.
- src/router/index.ts:114-123 y src/router/middleware.ts:18-21, 31, 87-94 contienen next() sin retorno consistente.

5.3 Creacion de ordenes de compra y venta

Estado: Parcial

* La forma de la peticion esta alineada, pero la validacion semantica de campos criticos es debil tanto en frontend como en backend.

* El store de compra envia red_id, shipping_type_id, wallet_id y address_send al endpoint order.
* OrderController persiste esos mismos campos, lo que demuestra que el contrato estructural si existe.
* El problema es que StoreOrderRequest no exige red_id, shipping_type_id ni address_send, y la vista de confirmacion de compra anula la validacion real devolviendo true.
* La union entre capas, por tanto, acepta datos criticos sin cierre semantico fuerte sobre red, direccion y modalidad de envio.

Evidencia observable

- src/views/user/Compra/CompraStore.ts:17, 26, 31 y 32 declara red_id, shipping_type_id, wallet_id y address_send; en 49 publica order.
- src/views/user/Compra/ConfirmOrder.vue:222-223 define validateAddress y devuelve true.
- Xelergo-Backend/app/Http/Controllers/Order/OrderController.php:233, 239, 242 y 245 persiste wallet_id, red_id, shipping_type_id y address_send.
- Xelergo-Backend/app/Http/Requests/Order/StoreOrderRequest.php:28-37 valida wallet_id, currency_id y payment_type_id, pero no exige red_id, shipping_type_id ni address_send.

5.4 Wallets y direcciones de deposito

Estado: Parcial

* El backend si crea direcciones con proveedor, y el frontend consume parte de esa capacidad, pero la abstraccion de proveedor no esta cerrada de forma consistente.

* El frontend solicita client/wallet/adress/wallet para obtener direcciones de wallet necesarias en el flujo de orden.
* ClientWalletController decide entre BitGo y un proveedor alterno para crear direcciones segun platform_crypto_id.
* Aun asi, getAddress consulta siempre BitGo, incluso cuando la direccion pudo generarse con el proveedor alterno, lo que rompe la uniformidad del contrato.
* Adicionalmente, el modulo XCOP de recepcion genera un QR interno basado en user_id, username y amount; eso describe un identificador interno entre usuarios, no una direccion cripto externa del proveedor.

Evidencia observable

```
- src/views/user/Compra/CompraStore.ts:88 llama client/wallet/adress/wallet.
- Xelergo-Backend/app/Http/Controllers/ClientWallet/ClientWalletController.php:259-271 crea direcciones via BitGo o CreateAdressUSDTCrytoCurrency segun platform_crypto_id.
- Xelergo-Backend/app/Http/Controllers/ClientWallet/ClientWalletController.php:296-300 consulta siempre bitgo->GetAdress().
- src/views/user/walletXCOP/Received/ReceivedComponent.vue:61-64 serializa user_id, username y amount dentro del QR.
```

5.5 Transferencia interna XCOP

Estado: Alineado

* La transferencia XCOP entre usuarios del propio sistema esta correctamente cerrada en su alcance interno.

* El frontend envia user_receipt_id y xcop_send al endpoint users/send/xcop.
* El backend valida ambos datos, impide autoenvio, verifica saldo, actualiza balances y registra un shipping record con tranx_no.
* Este flujo si constituye un contrato observable coherente entre interfaz, request validator y logica transaccional.
* No debe confundirse con un envio cripto externo: es una transferencia interna de saldo XCOP en el ecosistema de la aplicacion.

Evidencia observable

- src/views/user/walletXCOP/Send/SendComponent.vue:227-230 publica users/send/xcop con user_receipt_id y xcop_send.
- Xelergo-Backend/app/Http/Requests/User/UserRequest.php:25-26 exige user_receipt_id y xcop_send.
- Xelergo-Backend/app/Http/Controllers/User/UserController.php:809-839 valida saldo, evita autoenvio y mueve balances.

5.6 Retiro XCOP

Estado: Parcial

* El backend soporta mas capacidad que la que la UI realmente expone, por lo que el contrato visible al usuario queda incompleto.

* La pantalla de retiro enlaza SelectPaymentMethod solamente a form.account_id y fija type en Cuenta.
* El backend acepta tanto wallet_id como account_id para withdrawal/xcop.
* Como la UI no envia wallet_id, la ruta wallet-based del backend no forma parte del contrato real del frontend.
* El retiro termina ademas en procesamiento administrativo, no en payout automatico por proveedor cripto.

Evidencia observable

- src/views/user/walletXCOP/Swap/SwapComponent.vue:47 enlaza SelectPaymentMethod a form.account_id; 80-81 inicializa type='Cuenta' y wallet_id=''.
- src/views/user/walletXCOP/Swap/SwapComponent.vue:100-102 publica withdrawal/xcop con account_id.
- Xelergo-Backend/app/Http/Requests/WithdrawalXcop/StoreWithdrawalXcopRequest.php:25-26 permite wallet_id y account_id.
- Xelergo-Backend/app/Http/Controllers/WithdrawalXcop/WithdrawalXcopController.php:97-98 persiste ambos campos si llegan.

5.7 Estados en tiempo real de ordenes

Estado: No alineado

* Hay infraestructura de eventos, pero el contrato real entre broadcasting, suscripcion privada y representacion visual no es confiable.

* El backend emite eventos de orden tanto en PrivateChannel('orders.{id}') como en Channel('orders.{id}').
* El frontend, sin embargo, se suscribe a private-orders.{id} y depende de broadcasting/auth.
* BroadcastServiceProvider si expone broadcasting/auth con auth:sanctum, pero la autorizacion del canal privado orders.{id} esta comentada en routes/channels.php.
* El componente CheckBuy recrea suscripciones cada 30 segundos y almacena mensajes en refs que no gobiernan directamente la UI visible.
* El timeline mostrado al usuario usa textos y tiempos estaticos, no un lifecycle respaldado por estados reales del backend.

Evidencia observable

```
- src/pusher.ts:18-27 construye authEndpoint desde VITE_API_URL, usa Bearer y se suscribe con Pusher.
- Xelergo-Backend/app/Providers/BroadcastServiceProvider.php:15-17 registra Broadcast::routes con auth:sanctum.
- Xelergo-Backend/routes/channels.php:22-26 deja comentada la autorizacion del canal orders.{id}.
- Xelergo-Backend/app/Events/orderStatusEvent.php:31-32 y orderStatusChangeEvent.php:37-38 emiten a canales privado y publico.
- src/views/user/Venta/CheckBuy/CheckBuy.vue:41-53 y 65-74 recrea new usePusher sobre private-orders.{id}.
- src/views/user/Timeline/TimelineView.vue:169-185 muestra estados estaticos y en 208 recarga toda la pagina.
```

5.8 Flujo administrativo con txid manual

Estado: Parcial

* Existe un puente administrativo entre frontend y backend para aceptar operaciones por txid, pero no corresponde a un contrato automatizado con proveedor externo.

* La interfaz administrativa solicita un txid manual y lo envia a order/external/accept/deposit.
* El backend responde aceptando la orden y guardando ese txid.
* Eso puede ser util como operacion asistida, pero no sustituye un settlement real ni una confirmacion programatica contra proveedor o blockchain.
* Por diseno, este flujo confirma que parte del lifecycle cripto todavia depende de intervencion manual.

Evidencia observable

```
- src/views/admin/transactions/TransactionDetail/QrBuy.vue:42, 80-88 solicita txid y publica order/external/accept/deposit.
- Xelergo-Backend/routes/api.php:980-981 vincula external/accept/deposit con externalDisbursement.
- Xelergo-Backend/app/Http/Controllers/Order/OrderController.php:848, 856-857 define externalDisbursement, marca status aceptado y guarda txid.
```

6. Seguridad y robustez transversal

Lectura ejecutiva de seguridad

* La seguridad backend protege muchas rutas con sanctum, pero el contrato cliente-servidor sigue exponiendo riesgos evitables.

* El principal problema no es un unico bug aislado, sino la suma de decisiones debiles en token storage, consumo de terceros, errores y datos sensibles.

* El token vive en localStorage y helper.ts lo adjunta automaticamente a cualquier peticion, incluidas URLs absolutas externas.
* WalletStore y FeesStore consumen https://ticker.xeler.io y https://api.coinbase.com a traves del mismo helper, por lo que el Bearer puede viajar a terceros si el usuario esta autenticado.
* UserResource expone explicitamente google2fa_secret aunque el modelo User lo oculta por serializacion estandar; showUserActive devuelve ese resource y el frontend lo consume en updateUserAuth.
* La vista de confirmacion de compra usa v-html para pintar textos de opciones; hoy esos textos nacen del codigo propio, pero el patron no es seguro si manana pasan a ser dinamicos.
* El helper de errores asume error.response.data y luego error.response.data.data.errors, con riesgo de excepcion secundaria cuando la red cae antes de recibir respuesta.

Evidencia observable

```
- src/helper.ts:56 inyecta Authorization Bearer para cualquier request; 70 y 83
dereferencia error.response.data sin blindaje completo.
- src/stores/WalletStore.ts:29-31 consulta https://ticker.xeler.io/v1/ticker via
helper.http(url).
- src/stores/FeesStore.ts:98-99 consulta https://api.coinbase.com/v2/prices/BTC-USD/
spot via helper.http(url).
- Xelergo-Backend/app/Http/Resources/UserResource.php:32 expone google2fa_secret.
- Xelergo-Backend/app/Http/Controllers/User/UserController.php:402 y 433 entrega
UserResource desde showUserActive; src/stores/UserStore.ts:58 consume users/activite/
user.
- src/views/user/Compra/ConfirmOrder.vue:143 usa v-html para renderizar texto de
comisiones.
```

7. Estado del objetivo principal: cripto por proveedor externo

La pregunta central de este contrato no es si existen pantallas de compra, venta, retiro o recepcion, sino si el sistema completo soporta de manera verificable el flujo principal de envio y recepcion de criptomonedas por proveedor externo. La respuesta, con la evidencia combinada de frontend, backend y auditorias previas, es negativa.

Conclusion contractual del objetivo principal

* No esta correctamente soportado end-to-end.

* Si existen piezas parciales: creacion de direcciones, consultas de wallet, eventos de orden y algunos puntos de UI.

* Lo que no existe de forma cerrada es el lifecycle operativo completo: recepcion confirmada por proveedor, deteccion confiable, representacion visual fiel, payout real y conciliacion automatica.

* La auditoria backend ya concluia que el objetivo principal no cumple; la revision de uniones confirma que el frontend tampoco logra cerrar ese faltante.
* ClientWalletController crea direcciones con proveedor, pero DetectOrderNetworkCommand cambia estados internamente sin consultar blockchain ni proveedor; el frontend luego presenta timelines estaticos, lo que amplifica la falsa sensacion de cierre.
* El supuesto desembolso externo del backend sigue dependiendo de un txid manual; la UI administrativa refleja exactamente ese enfoque asistido, no una integracion de settlement.
* La recepcion XCOP del frontend es un QR interno entre usuarios, no una direccion on-chain del proveedor. Eso sirve para transferencias internas, no para demostrar soporte cripto externo.

Evidencia observable

- Xelergo-Backend/AUDITORIA_QA_PROYECTO.html:164, 170 y 439-463 ya documenta que el backend no cumple el objetivo principal por proveedor externo.
- Xelergo-Backend/app/Console/Commands/DetectOrderNetworkCommand.php:36, 40 y 45 cambia status_id de 1 a 2 y notifica 'detectada en red' sin consultar proveedor externo.
- Xelergo-Backend/app/Services/BitgoService.php:97-103 define sendTransaction() pero ejecuta un GET al endpoint de address.
- src/views/user/Timeline/TimelineView.vue:169-185 muestra estados estaticos y tiempos calculados localmente.
- src/views/admin/transactions/TransactionDetail/QrBuy.vue:80-88 y OrderController.php:856-857 sostienen el flujo manual por txid.

8. Brechas priorizadas y contrato minimo requerido

Acciones obligatorias antes de considerar aprobacion

* Corregir los endpoints sensibles con slash inicial y cerrar el contrato real de confirmacion post-login.
* Definir y aplicar una sola estrategia de canal para ordenes: o se autoriza el canal privado correctamente o el frontend se adapta deliberadamente al canal publico.
* Reinstalar validacion real de address_send, red_id y shipping_type_id en frontend y backend.
* Quitar el Bearer de peticiones hacia terceros y separar claramente clientes HTTP internos de clientes externos.
* Eliminar la exposicion de google2fa_secret en UserResource.
* Cerrar el flujo cripto real por proveedor o declarar formalmente que el producto opera solo con procesos asistidos.

* El retiro XCOP debe decidirse contractual y visualmente: o se soporta retiro a wallet y a cuenta, o se elimina del backend la capacidad que la UI no ofrece.
* La representacion de estados debe depender de senales backend verificables y no de textos estaticos ni recargas completas de pagina.
* La observabilidad minima debe incluir pruebas de contrato, pruebas de integracion y escenarios de fallo para ordenes, wallets, broadcasting y XCOP.

* Si se mantiene una operacion asistida por backoffice, debe documentarse como tal y no presentarse en la UI como si fuera un flujo cripto automatizado por proveedor.

9. Decision final

El estado actual del contrato tecnico end-to-end puede describirse como parcialmente funcional y estructuralmente fragil. El producto si tiene bloques reutilizables y varias uniones operativas entre frontend y backend, pero su capa visual transmite mayor madurez que la que realmente existe en la integracion.

Desde perspectiva ejecutiva, el sistema no deberia aprobarse como solucion lista para soportar envio y recepcion de criptomonedas por proveedor externo. Desde perspectiva tecnica, lo que hoy existe se parece mas a una V1 asistida, con piezas manuales y validaciones incompletas, que a una arquitectura confiable para evolucionar sin romper otros flujos.

Dictamen

* Viabilidad actual: solo parcial y con alto cuidado operativo.
* Aprobacion general: no.
* Estado del objetivo principal cripto por proveedor externo: no soportado correctamente end-to-end.
* Arquitectura para evolucion: reutilizable en partes, pero hoy demasiado acoplada y con riesgo de regresion si se expande sin antes cerrar contratos.

Creado por Jose Manuel Guerrero Duarte, Ing en informatica